

STM32G431 FOC 技术手册

电机参数持久化与 无刷电机音乐功能

代码原理、接口使用、换歌、分辨率调节与故障排查

适用工程: g431_foc | MCU: STM32G431CBUx

版本: 2026-06-21 | 已通过 Debug 全量编译与链接检查

阅读说明与安全边界

重要安全说明 参数辨识会主动拖动并对齐转子，约持续 5.4 秒。仅在电机卸载、外壳固定、运动范围无干涉时调用；带大惯量负载时不要辨识。播放音乐也会产生交变转矩，应拆除桨叶和危险负载，并从较小 Iq 幅值开始。

本文按当前工作区实际代码编写，覆盖两项新增能力：参数辨识结果持久化，以及利用 FOC q 轴交变电流让无刷电机发声。文中的接口、常量、时序和限制均对应本次编译通过的实现。

一分钟快速使用

- 1. 首次烧录后观察 OLED：如果显示 NoParam，表示 Flash 中尚无有效参数，电机保持零力矩。
- 2. 卸下负载并在一次性事件中调用 Motor_Parameters_IdentifyAndSave(); 等待辨识、校验和 Flash 写入完成。
- 3. 状态变为 MOTOR_PARAMETERS_READY 后，可调用 Motor_Music_StartDemo() 播放一次示例曲。
- 4. 后续上电直接读取 Flash 参数，不再自动辨识；需要重新标定时才再次调用主动辨识函数。

```
#include "motor_parameters.h"
#include "motor_music.h"

// 必须由按键沿、串口命令等一次性事件触发，不能每次 while 都调用。
if (need_identify) {
    (void)Motor_Parameters_IdentifyAndSave();
}

// 参数就绪后播放示例曲 3 次。
if (Motor_Parameters_IsReady()) {
    Motor_Music_StartDemoTimes(3U);
}
```

1. 代码组成与职责

文件	职责
motor_parameters.c/.h	Flash 参数记录、CRC/范围校验、主动辨识编排、保存状态机

文件	职责
motor_identify.c/.h	电阻、相序、极对数、零位辨识；加载已验证参数
motor_music.c/.h	音符调度、相位累加、波形插值、播放次数与音量控制
motor_songs.c/.h	独立曲谱文件；内置《小星星》，推荐在此增加或替换曲谱
motor_system.c	1 kHz 总调度；播放时暂停位置/速度环；参数就绪才允许闭环
motor_current_loop.c	20 kHz 电流环入口；音乐模式下替换 pi_q.target
hardware_init.c / main.c	上电加载 Flash；主循环执行非实时 Flash 保存任务
STM32G431XX_FLASH.ld	应用 Flash 限制为 126 KB；最后 2 KB 专用于参数
CMakeLists.txt / tasks.json	编入新增模块；pyOCD 强制 sector 擦除以保留参数页

设计原则是把曲谱、播放器、参数存储和总控制调度分开。换歌不需要修改 FOC；重新辨识不需要修改 Flash 驱动；音乐停止后正常控制链自动恢复。

2. 总体运行架构

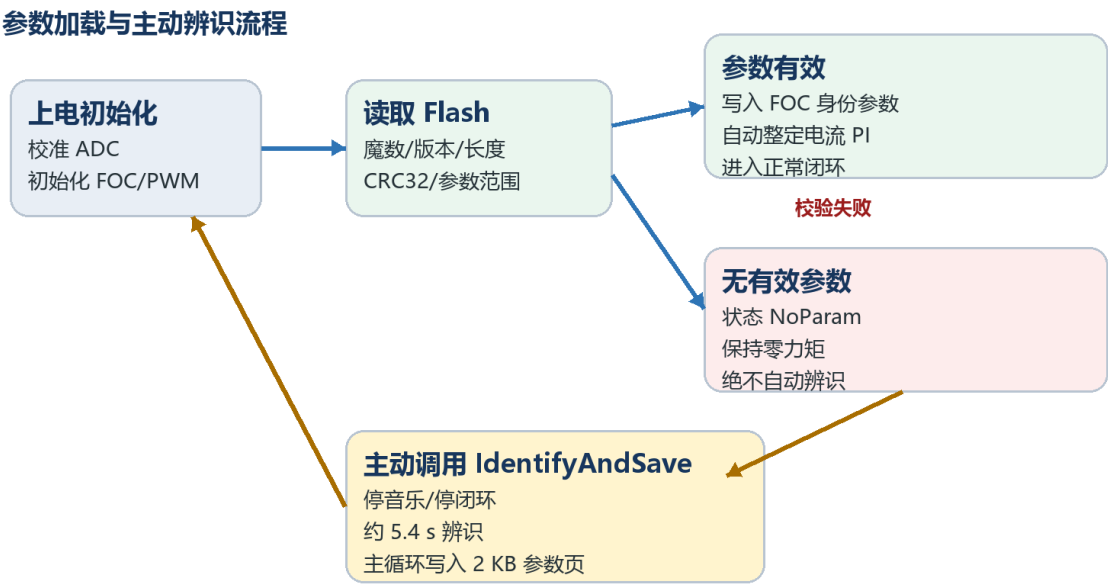


图1 参数加载与主动辨识流程

上电路径不包含 Motor_Identify_Start()。系统只读取参数页并校验；校验失败时状态为 MOTOR_PARAMETERS_NO_DATA，FOC 和 SVPWM 不会进入有力矩的闭环状态。

3. 参数持久化功能

3.1 上电加载与状态机

状态	含义	系统行为
NO_DATA	无有效记录	输出关闭；OLED 显示 NoParam；等待人工触发辨识
READY	参数有效	参数写入 FOC，自动整定电流 PI，允许正常控制和音乐
IDENTIFYING	正在辨识	关闭闭环/音乐，TIM2 每 1 ms 推进辨识状态机
SAVE_PENDING	等待保存	电机保持零力矩；主循环执行 Flash 擦写
ERROR	辨识或保存失败	输出保持关闭；OLED 显示 ParamErr

3.2 辨识过程与精度调整

阶段	时间/行程	实现说明
电阻 R	400 ms 稳定 + 600 ms 平均	1 kHz 读取滤波后的相电流；由 $R=U/I$ 计算
电感 L	100 ms 退磁等待	当前仍采用厂家标称相电感 0.6 mH，不是实测值
相序/极对数	1 s 锁定 + 3 电周期慢速扫描	累计 AS5600 机械位移，判断 uvw_dir 并计算极对数
零位	2 s 对齐 + 500 ms 圆周平均	对 sin/cos 求平均，可正确处理 0/4096 跨界

精度边界 延长读取时间可以降低电流噪声、编码器量化和机械晃动造成的误差，但无法把固定的 0.6 mH 变成实测电感。若更换电机型号，应先确认厂家相电感。

3.3 Flash 记录与校验

链接脚本把 STM32G431CBUx 的 128 KB Flash 拆为 126 KB 应用区和末尾 2 KB 参数页。参数页起始地址为 0x0801F800，普通固件段不会覆盖它。

字段	格式	作用
magic	0x4D504152	识别这是电机参数记录
version / record_size	版本 1 / 结构长度	拒绝不兼容格式
resistance / inductance	float	电流环自动整定输入
zero_angle_offset	float	机械零位偏置, 范围 $[0, 2\pi)$
pole_pairs / uvw_dir	整数	极对数 1..64; 方向必须为 +1 或 -1
crc32	CRC-32	检测掉电、擦写失败和随机数据

写入过程只在主动辨识完成后执行。Flash 擦写放在 main() 主循环的

Motor_Parameters_BackgroundTask() 中, 不占用 1 ms 实时中断。擦写期间电流环和 SVPWM 已关闭。

3.4 主动辨识接口怎么用

```
#include "motor_parameters.h"

// 返回 1: 请求被接受; 返回 0: 当前正处于辨识或保存阶段。
uint8_t accepted = Motor_Parameters_IdentifyAndSave();

MotorParametersStatus status = Motor_Parameters_GetStatus();
if (status == MOTOR_PARAMETERS_READY) {
    // 参数已加载或新辨识结果已成功保存, 可以运行电机。
}
```

5. 函数会先停止音乐、清零 d/q 目标、关闭电流环和 SVPWM, 再启动辨识。
6. 不要在 while(1) 中无条件反复调用, 应由按键上升沿、串口命令或维护模式触发一次。
7. 辨识失败时旧参数页不会主动用于当前运行; 重新上电仍可重新校验原记录。
8. VS Code 的两个 DAPLink 任务均显式使用 -e sector; 不要改成 --erase chip, 否则参数页会被清空。

4. 无刷电机音乐的实现原理

4.1 为什么无刷电机能发声

FOC 中 q 轴电流主要产生转矩。播放器把一个均值约为 0 的正弦波作为 q 轴目标电流，使电磁转矩按音频频率正负交替；定子、转子和安装结构被周期性激励后产生可听振动。d 轴目标保持为 0，因此音乐功能没有主动注入励磁电流。

不是扬声器 实际响度和音色高度依赖电机结构、安装方式、负载、母线电压以及 200 Hz 电流环带宽。相同曲谱在不同电机上的声音不会完全一致。

4.2 两个时间尺度

音乐播放的双时基控制链

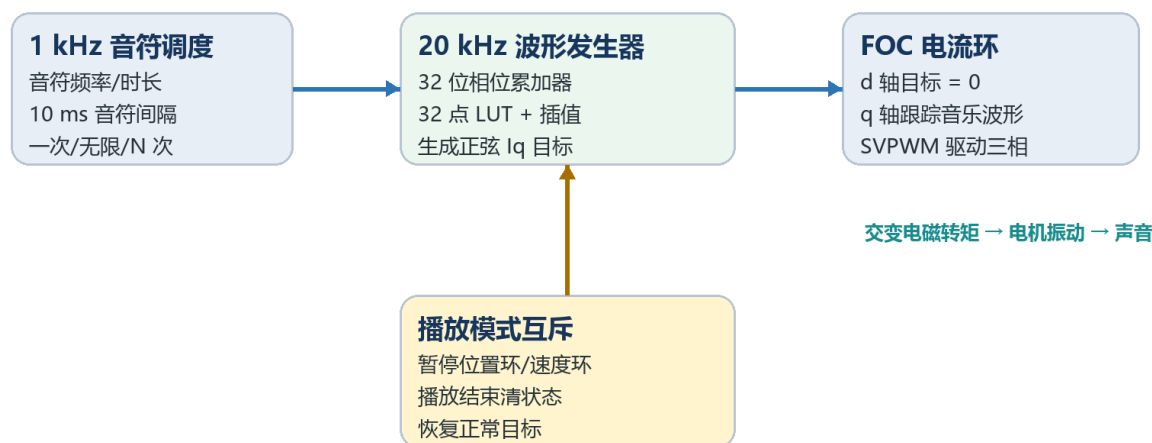


图2 音符调度与 20 kHz 波形发生链

9. 1 kHz：Motor_Music_Task1ms() 管理音符剩余毫秒数、换音、10 ms 收尾静音和重复次数。

10. 20 kHz：Motor_Music_ProcessIqTarget() 在 ADC 注入转换回调所处的电流环内生成每个正弦采样点。

11. 播放时：motor_system.c 暂停速度 PID、位置 PID 和轨迹规划，避免它们与交变音乐转矩互相对抗。
12. 未播放时：ProcessIqTarget() 原样返回 normal_iq_target，正常位置/速度控制路径不变。

4.3 相位累加器与正弦插值

播放器使用 32 位相位累加器。每个 20 kHz 周期增加的相位步长为：

$$\text{phase_step} = \text{frequency_hz} \times 2^{32} / 20000$$

相位高 5 位选择 32 点正弦表中的当前点；默认再取 8 位做相邻点线性插值，因此 LUT 只占 128 字节，却具有 $32 \times 256 = 8192$ 个等效相位位置。注意：每秒实际仍只输出 20000 个采样点，插值不会提高真实采样率。

5. 曲谱格式与换歌

5.1 一个音符包含什么

```
typedef struct
{
    float frequency_hz;    // 音高, 可使用 440.5f 等小数频率; 0 表示休止
    uint16_t duration_ms;  // 时长, 单位 ms; 最小有效值按 1 ms 处理
} MotorMusicNote;
```

常用音名宏位于 motor_music.h，例如 MOTOR_NOTE_C4=261.63 Hz、MOTOR_NOTE_A4=440.00 Hz。没有宏的升降音可直接填写频率。每个大于 20 ms 的有声音符末尾自动留出 10 ms 静音，以区分连续同音。

5.2 最简单的换歌方式

内置曲谱已从播放器拆到 user/src/motor_songs.c。只改曲谱数组即可，不要改相位发生器。

```
// user/src/motor_songs.c
const MotorMusicNote g_motor_song_twinkle[] = {
    {MOTOR_NOTE_C4,      300U},
    {MOTOR_NOTE_E4,      300U},
    {MOTOR_NOTE_G4,      500U},
    {MOTOR_MUSIC_REST,    120U},
    {440.50f,             400U}, // 允许自定义小数 Hz
```

```
};

const uint16_t g_motor_song_twinkle_count =
    (uint16_t)(sizeof(g_motor_song_twinkle) /
               sizeof(g_motor_song_twinkle[0]));
```

数组生命周期 Motor_Music_Start*() 只保存曲谱指针，不复制曲谱。曲谱必须是全局/static const 数组，不能把函数内的普通局部数组传入后立即返回。

5.3 增加第二首歌

推荐在 motor_songs.c/h 中增加新的全局只读数组和计数，而不是覆盖示例曲。

```
// motor_songs.h
extern const MotorMusicNote g_motor_song_scale[];
extern const uint16_t g_motor_song_scale_count;

// motor_songs.c
const MotorMusicNote g_motor_song_scale[] = {
    {MOTOR_NOTE_C4, 180U}, {MOTOR_NOTE_D4, 180U},
    {MOTOR_NOTE_E4, 180U}, {MOTOR_NOTE_F4, 180U},
    {MOTOR_NOTE_G4, 180U}, {MOTOR_NOTE_A4, 180U},
    {MOTOR_NOTE_B4, 180U}, {MOTOR_NOTE_C5, 360U},
};
const uint16_t g_motor_song_scale_count =
    sizeof(g_motor_song_scale) / sizeof(g_motor_song_scale[0]);
```

6. 播放控制：一次、无限和指定次数

目的	调用	说明
播放一次	Motor_Music_Start(song, count, 0U);	曲末自动停止并恢复正常控制
无限循环	Motor_Music_Start(song, count, 1U);	直到调用 Motor_Music_Stop()
精确 N 次	Motor_Music_StartTimes(song, count, N);	N=3 即完整播放三遍
示例曲一次	Motor_Music_StartDemo();	内置《小星星》
示例曲 N 次	Motor_Music_StartDemoTimes(N);	便于快速验证
立即停止	Motor_Music_Stop();	下一次 1 ms 调度恢复外环

```
#include "motor_music.h"
#include "motor_songs.h"

// 播放三次
Motor_Music_StartTimes(g_motor_song_twinkle,
```



```
        g_motor_song_twinkle_count,
        3U);

// 无限循环
Motor_Music_Start(g_motor_song_twinkle,
                  g_motor_song_twinkle_count,
                  1U);

// 在按键或串口停止命令中调用
Motor_Music_Stop();
```

play_count=0 会直接停止，不代表无限循环。无限循环仍使用兼容接口 Motor_Music_Start(..., 1U)。这样旧代码保持不变，同时新增精确次数控制。

7. 音乐分辨率怎么调

“音乐分辨率”可能指音高精度、波形平滑度或节奏精度。这三者由不同参数控制，不能只改一个宏就同时提高。

维度	当前实现	解释
音高输入精度	float frequency_hz	可写 440.5f；优于原整数 Hz
NCO 理论步进	$20000 / 2^{32} \approx 0.00000466 \text{ Hz}$	由 32 位相位累加器决定
波形插值精度	MOTOR_MUSIC_INTERPOLATION_BITS	默认 8 位；影响正弦阶梯感与谐波
节奏精度	Motor_Music_Task1ms()	时长分辨率为 1 ms
真实采样率	20 kHz ADC/电流环	每秒最多产生 20000 个 Iq 采样
频率上限	MOTOR_MUSIC_MAX_FREQUENCY_HZ=4000	保守限制；高音通常受电流环/机械结构衰减

7.1 调波形插值位数

INTERPOLATION_BITS	等效相位位置	建议
0	32	无插值，CPU 最省，阶梯和高次谐波最多
4	512	轻量平滑
8（默认）	8192	平滑度与运算量的平衡点

INTERPOLATION_BITS	等效相位位置	建议
12	131072	理论更细，但听感提升可能很小

```
// user/inc/motor_music.h
#define MOTOR_MUSIC_INTERPOLATION_BITS 8U
```

插值位数只改变查表点之间的幅值估算精度，不改变 20 kHz 真实输出速率。STM32G431 带 FPU，默认 8 位插值的额外浮点开销较低；如果要改到更高值，应重新测量 20 kHz 电流环 ISR 余量。

7.2 不要单独修改采样率宏

同步约束 MOTOR_MUSIC_SAMPLE_RATE_HZ 必须等于 ADC 注入回调/电流环的真实频率。当前 TIM1、电流 PID dt=50 μs 和音乐宏共同对应 20 kHz。只改音乐宏会让所有音高按比例跑偏。

若确实要修改真实采样率，必须同时检查 TIM1 计数周期、ADC 触发边沿、电流环 PID 的 dt、ADC 滤波截止频率、CPU 中断占用以及 MOTOR_MUSIC_SAMPLE_RATE_HZ。此项属于电流环架构变更，不建议为了普通换歌而调整。

7.3 音高、时值和音量

- 13. 音高：直接写 float Hz；十二平均律可按 $f=440 \times 2^{(n-69)/12}$ 计算。
- 14. 时值：duration_ms 为整数毫秒；例如 120 BPM 下四分音符约 500 ms。
- 15. 音量：Motor_Music_SetAmplitude(0.08f) 设置 q 轴峰值电流，单位 A。
- 16. 默认幅值 0.12 A，代码硬上限 0.30 A；音量并不与听感线性对应。

```
Motor_Music_SetAmplitude(0.08f); // 首次测试建议更小
Motor_Music_StartDemoTimes(2U);
```

8. 自动播放与事件触发

8.1 上电自动播放示例曲

默认关闭，避免电机在无人确认时突然振动。若需要参数加载成功后自动播放一次：

```
// user/inc/motor_music.h
#define MOTOR_MUSIC_AUTOPLAY_DEMO 1
```

自动播放只有在 Motor_Parameters_IsReady() 为真时触发；无 Flash 参数时不会绕过安全门槛。当前自动播放调用 Motor_Music_StartDemo(), 因此每次上电只播放一遍。

8.2 按键/串口触发的正确模式

```
static uint8_t last_button = 0U;
uint8_t now = ReadMusicButton();

if ((now != 0U) && (last_button == 0U) &&
    Motor_Parameters_IsReady()) {
    Motor_Music_StartDemoTimes(3U); // 仅在上升沿触发一次
}
last_button = now;
```

不要在按键保持按下时每个循环都重新调用 Start；否则当前音符会不断被重置到第一拍。串口命令也应在收到一帧完整命令时调用一次。

9. 常见问题与定位

现象	可能原因	处理
上电不转, OLED=NoParam	参数页为空或校验失败	卸载电机后主动调用 IdentifyAndSave
OLED=ParamErr	辨识异常或 Flash 写入失败	检查 AS5600、电流采样、电源；断电重试 前先排故
调用播放但无声	参数未 READY、幅值过小或电机未固定	查状态；先用 0.08~0.12 A；固定外壳
高音明显较弱	200 Hz 电流环和机械结构衰减	优先使用 C4~A4；不要直接提高电流环带宽
音高整体按比例偏移	采样率宏与真实 20 kHz 不一致	恢复 SAMPLE_RATE_HZ=20000
曲子只响第一拍	反复调用 Start 重置播放器	改为按键沿/单次串口事件触发
循环无法结束	使用了 repeat=1 无限模式	调用 Stop，或改用 StartTimes
重新烧录后参数丢失	使用了 chip erase	使用工程的 sector 烧录任务并重新辨识
电机明显来回摆动	低频/幅值过大或负载惯量大	提高最低音、降低幅值、卸载测试

10. 当前限制与本次复核改进

10.1 本次发现并已修正

17. 频率字段由 `uint16_t` 改为 `float`，可使用标准音高的小数频率。
18. 32 点正弦 LUT 增加可配置线性插值，默认 8 位插值。
19. 新增 `Motor_Music_StartTimes()` 与 `StartDemoTimes()`，可精确播放 N 次。
20. 曲谱拆分为 `motor_songs.c/.h`，换歌不再修改播放引擎。

10.2 仍然存在的工程限制

21. 电感是 0.6 mH 厂家标称值，不是在线辨识结果。
22. 播放器为单声道单音，同一时刻只生成一个频率，不支持和弦。
23. 参数 Flash 使用单页记录和 CRC，没有双页掉电事务或磨损均衡；不应高频反复辨识保存。
24. 音乐电流经过 200 Hz 电流环，高频音符的实际电流幅值可能明显衰减。
25. 本次完成编译、链接区与静态路径检查，但未在实体电机上执行 Flash 擦写和声学测试。

11. 构建、烧录与验收

```
cmake --build --preset Debug
```

当前构建结果：RAM 约 3.8 KB / 32 KB；应用 Flash 约 74.9 KB / 126 KB；参数页固定为 0x0801F800~0x0801FFFF。输出文件为 `build/Debug/g431_foc.elf`。

建议的板级验收顺序

26. 首次烧录后确认 OLED 显示 `NoParam`，电机无力矩且不自行辨识。
27. 卸载状态调用 `IdentifyAndSave`，记录总时长、极对数、方向、R 和零位。
28. 断电重启，确认直接进入 `READY`，且不发生辨识动作。
29. 按 `sector` 方式重新烧录同一 ELF，确认参数仍保留。

30. 以 0.08 A 播放 C4/A4 单音，检查频率与温升，再播放示例曲。
31. 分别验证播放一次、三次和无限循环后 Stop。
32. 观察 VOFA CH7：音乐期间应看到实际送入 q 轴电流环的正弦目标。

完成标准 参数断电保持、上电不自动辨识、音乐播放次数正确、停止后位置控制恢复、连续短时播放无异常温升，才适合继续接入实际负载。